

Combining Parimutuel Settlement with Automated Market Makers for Liquidity-Guaranteed Prediction Markets: The Onix Protocol

Anatoly Piskunov*

2026

Abstract. Prediction markets aggregate dispersed information into probabilistic forecasts that consistently outperform polls and expert panels, yet their adoption is constrained by a structural liquidity problem: providers of market depth bear adverse selection risk that deters retail participation. We present the **Onix Protocol**, a hybrid architecture that decouples the *pricing* function from the *settlement* function in prediction markets. By pairing automated market maker pricing — a Constant Product Market Maker (CPMM) for binary outcomes and a Logarithmic Market Scoring Rule (LMSR) for multi-outcome markets — with parimutuel (totalizator) settlement, we achieve a **structural guarantee** that liquidity-provider principal is never at risk from betting outcomes. We formalize the protocol’s economic invariants, prove the LP principal guarantee for both market types, and resolve the central tension of any curve-priced market — that an early exit through the pricing curve realizes a trading P&L against the very LPs the protocol protects — with a *deferred outcome-contingent claim* mechanism, for which we prove a second no-mint / LP-safe invariant. We describe a “Lazy” liquidity pool enabling passive retail participation, analyze the dispute resolution mechanism under DAO governance, and discuss the experimental hypotheses this system is designed to test. The protocol is implemented as consensus-level operations on the VIZ distributed ledger.

KEY WORDS

1. prediction markets
2. automated market makers
3. parimutuel betting
4. LMSR
5. liquidity provision
6. DAO governance
7. mechanism design

*Independent Researcher — VIZ Distributed Ledger. Email: anatoly.piskunov@gmail.com. ORCID: 0009-0000-8883-4111.

0. Contents

1	Introduction	4
1.1	The Prediction Market Liquidity Problem	4
1.2	Research Questions	4
1.3	Contributions	4
2	Related Work	5
2.1	Market Scoring Rules	5
2.2	Constant Product Market Makers	5
2.3	Conditional Token Frameworks	5
2.4	Parimutuel Betting	5
2.5	Our Approach	5
3	Protocol Model	6
3.1	Participants and Roles	6
3.2	Market Definition	6
3.3	Fee Structure	6
3.4	Market State Machine	6
4	Pricing Mechanisms	7
4.1	Binary Markets: Constant Product Market Maker	7
4.2	Multi-Outcome Markets: LMSR	7
5	Parimutuel Settlement and LP Guarantee	8
5.1	Unified Settlement Model	8
5.2	Theorem: LP Principal Guarantee	8
5.3	Time Penalty on Late Bets	9
5.4	LP Time-Weighted Fee Distribution	10
6	“Lazy” Liquidity Pool	10
6.1	Motivation	10
6.2	Pool Mechanics	10
6.3	Reward Distribution (Lazy Accounting)	11
6.4	Opportunity-Cost Protection	11
6.5	Emergency Withdrawal	11
6.6	Opt-In Leverage Funded by the Pool	12
6.7	Design Decision: Real Depth Only (No Virtual/Phantom Liquidity)	13
7	Early Exit via Deferred Outcome-Contingent Claims	14
7.1	The Curve-Exit / Parimutuel Tension	14
7.2	The Deferred-Claim Mechanism	14
7.3	Settlement Distribution	15
7.4	Theorem: Bounded Early Exit, No Uncovered Shortfall	15
7.5	Economic Interpretation: A Bounded Liquidity Discount	16

7.6	Governance Parameter and Implementation	16
8	Dispute Resolution Under DAO Governance	17
8.1	Bonded Oracle Model	17
8.2	Two-Mode Dispute System	17
8.3	Dispute Outcomes and Incentive Compatibility	18
8.4	Game-Theoretic Analysis	18
8.5	No-Contest Declaration	19
9	The VIZ Token in the Prediction Market Experiment	19
9.1	Token Utility	19
9.2	Token Demand Feedback Loop	19
9.3	Experimental Value of the Token	20
10	Full Market Cycle: A Worked Example	20
10.1	Setup	20
10.2	Initialization	20
10.3	Betting Phase	20
10.4	Resolution	21
10.5	Dispute Scenario	21
10.6	Early-Exit Scenario	21
11	Anti-MEV: Batch and Commit-Reveal Betting	21
11.1	MEV in Prediction Markets	21
11.2	Batch Settlement	22
11.3	Commit-Reveal	22
12	Implementation	22
12.1	Consensus-Level Operations on VIZ DLT	22
12.2	On-Chain State	22
12.3	Governance Parameters	23
12.4	Scalability	23
13	Discussion	23
13.1	Comparison with Existing Approaches	23
13.2	Limitations and Honest Tradeoffs	24
13.3	Experimental Hypotheses	25
14	Conclusion	26

1. Introduction

1.1. The Prediction Market Liquidity Problem—Prediction markets are among the most reliable mechanisms for aggregating dispersed information into actionable probabilistic forecasts. Empirical evidence from political forecasting (Berg, Forsythe, Nelson & Rietz, 2008), sports betting (Levitt, 2004), and internal corporate markets (Cowgill, Wolfers & Zitzewitz, 2009) demonstrates that market prices consistently outperform alternative forecasting methods.

Despite this track record, prediction market deployment remains limited. The fundamental constraint is not demand for forecasts but *supply of liquidity*. Every existing prediction market architecture requires liquidity providers (LPs) to accept one of three risk profiles:

- (1) **Impermanent loss** in AMM-based markets (Adams, Zinsmeister & Robinson, 2020; Adams et al., 2021), where LPs systematically lose to informed traders;
- (2) **Adverse selection** in limit-order-book markets, where market makers bear inventory risk against better-informed counterparties;
- (3) **Bounded subsidy loss** in LMSR markets (Hanson, 2003), where the market maker risks up to $b \ln N$ against correctly-informed crowds.

These risk profiles restrict liquidity provision to sophisticated actors with active management capabilities, creating thin markets with high slippage that deter better participation — a self-reinforcing cycle:

thin markets → high slippage → poor UX → few bettors → low fees → no LP incentive → thin markets

1.2. Research Questions—This work poses and seeks to answer the following questions through a deployed experimental system:

Q1. *Can the pricing and settlement functions of a prediction market be architecturally separated such that LP principal is structurally insulated from betting outcomes?*

Q2. *Does a parimutuel settlement mechanism, where losers exclusively fund winners, eliminate the adverse selection problem inherent in AMM-based and LMSR-based prediction markets?*

Q3. *Can automated liquidity deployment via a pooled mechanism (“Lazy Pool”) reduce the barrier to LP entry sufficiently to bootstrap market depth without active market selection by individual LPs?*

Q4. *Is a DAO-governed dispute resolution mechanism, with stake-weighted public voting and bonded oracles, a viable alternative to centralized arbitration or optimistic oracle schemes?*

Q5. *What governance-token utility emerges from a prediction market protocol where LP positions carry voting weight, and does this create a sustainable token-demand feedback loop?*

1.3. Contributions—Our contributions are:

- (1) **A formal proof** that combining AMM pricing with parimutuel settlement yields a structural LP principal guarantee for both binary (CPMM) and multi-outcome (LMSR) prediction markets (Theorem 1).
- (2) **A deferred outcome-contingent claim mechanism** that resolves the curve-exit / parimutuel tension — allowing bettors and leveraged positions to exit early without realizing a trading P&L against LP depth — together with a proof (Theorem 2) that it cannot create an uncovered shortfall, mint tokens, or touch LP principal.
- (3) **The “Lazy Pool” mechanism** — an automated capital deployment system with graduated

recall and opportunity-cost protection that enables passive retail LP participation, plus an opt-in, default-off pool-funded leverage subsystem whose pool exposure is *bounded* and isolated from the Theorem 1 LP guarantee.

- (4) **A two-mode dispute resolution framework** combining bonded oracles with DAO committee arbitration, including a full game-theoretic analysis of oracle incentive compatibility.
- (5) **An experimental protocol** implemented as consensus-level operations on a production distributed ledger, with explicit hypotheses and measurable outcomes.

2. Related Work

2.1. Market Scoring Rules—Hanson (2003) introduced the Logarithmic Market Scoring Rule (LMSR), in which a market maker quotes prices via a softmax function over quantity parameters. The LMSR guarantees a bounded loss to the market maker of $b \ln N$ and ensures prices sum to unity. However, this bound is also the LP’s maximum risk — if the crowd correctly predicts the outcome, the market maker loses the entire subsidy. This has limited LMSR adoption to corporate environments (Microsoft, Inkling) where the operator absorbs losses.

2.2. Constant Product Market Makers—The constant product formula $x \cdot y = k$, popularized by Uniswap (Adams, Zinsmeister & Robinson, 2020), provides continuous pricing for two-asset pairs. When applied to binary prediction markets, the reserves represent opposing outcomes and the invariant provides natural probability pricing. However, standard AMM LPs suffer impermanent loss — empirical analysis shows over 50% of Uniswap v3 LPs underperform buy-and-hold (Adams et al., 2021).

2.3. Conditional Token Frameworks—Polymarket employs the Gnosis Conditional Tokens Framework (CTF), where positions are ERC-1155 tokens with split/merge operations to enforce price coherence ($\sum p_i = 1$). This adds composability but introduces complexity and requires an external arbitrage mechanism. UMA’s Optimistic Oracle provides dispute resolution through a challenge-response game with economic bonds.

2.4. Parimutuel Betting—The parimutuel (totalizator) model, originating from Pierre Oller’s totalisator (1865), pools all bets and distributes the losers’ stakes proportionally among winners. This guarantees the house never pays from its own capital. However, traditional parimutuel systems lack real-time pricing — odds are only final at pool close — making them unsuitable for continuous information aggregation.

2.5. Our Approach—The Onix Protocol combines the real-time pricing capability of AMMs/LMSR with the zero-LP-risk property of parimutuel settlement. The AMM or LMSR serves exclusively as a *pricing engine* — determining implied probabilities and assigning weights — while settlement follows the parimutuel model where losers fund winners and LP principal is architecturally separated from the payout flow.

This positions Onix differently from two other hybrid directions in the literature. First, *liquidity-sensitive* market makers (Othman, Pennock, Reeves & Sandholm, 2013; Abernethy, Chen & Vaughan, 2011) keep the operator as counterparty but bound or shape its loss via the cost-function geometry; the operator can still lose. Onix instead removes the counterparty role from the LP altogether — the curve only *prices*, it never *pays*. Second, *CLOB+AMM* hybrids (e.g. order books backstopped by an AMM, as explored in several DEX designs and in Polymarket’s CLOB

with AMM-style fallbacks) blend two pricing venues but inherit inventory/adverse-selection risk from the market-making side. Onix keeps a single continuous pricing venue and moves the risk-bearing entirely out of pricing and into the parimutuel pool. To our knowledge, pairing a continuous AMM/LMSR pricer with parimutuel (losers-fund-winners) settlement to obtain an unconditional LP-principal guarantee has not previously been formalized.

3. Protocol Model

3.1. *Participants and Roles*—We define the following participant set $\mathcal{P}$:

Role	Symbol	Function
Market Creator	$c \in \mathcal{P}$	Defines market parameters, provides seed liquidity
Oracle	$o \in \mathcal{P}$	Registers insurance bond I_o , accepts markets, resolves outcomes
Bettor	$b_i \in \mathcal{P}$	Places bets on outcomes, receives outcome tokens
Liquidity Provider	$\ell_j \in \mathcal{P}$	Deposits capital into market pools or the Lazy Pool
Dispute Resolver	$\mathcal{D}$	Committee (stake-weighted vote) or named account

3.2. *Market Definition*—A market $\mathcal{M}$ is a tuple:

$$\mathcal{M} = (o, \tau, \mathbf{O}, t_{\text{bet}}, t_{\text{res}}, \theta, L)$$

where o is the designated oracle, $\tau \in \{0, 1\}$ is the market type (binary or multi), $\mathbf{O} = \{O_1, \dots, O_N\}$ is the outcome set ($N = 2$ for binary, $3 \leq N \leq 10$ for multi), t_{bet} and t_{res} are betting and resolution expiration times, θ is the fee parameter vector, and L is the initial liquidity.

3.3. *Fee Structure*—All fees are expressed in basis points (bp), where 10000 bp = 100%. The fee vector is:

$$\theta = (\theta_{\text{oracle}}, \theta_{\text{creator}}, \theta_{\text{liq}})$$

Fees are computed exclusively at resolution from the losing side's aggregate stake S_{lose} and are never deducted at bet placement time:

$$\begin{aligned} f_{\text{oracle}} &= \lfloor S_{\text{lose}} \cdot \theta_{\text{oracle}} / 10000 \rfloor \\ f_{\text{creator}} &= \lfloor S_{\text{lose}} \cdot \theta_{\text{creator}} / 10000 \rfloor \\ f_{\text{liq}} &= \lfloor S_{\text{lose}} \cdot \theta_{\text{liq}} / 10000 \rfloor \\ W &= S_{\text{lose}} - f_{\text{oracle}} - f_{\text{creator}} - f_{\text{liq}} \end{aligned}$$

where W is the **winners' pool** available for distribution to winning bettors.

3.4. *Market State Machine*—The lifecycle of $\mathcal{M}$ follows a finite state machine:

$$q_0 \xrightarrow{\text{oracle accepts}} q_1 \xrightarrow{t \geq t_{\text{bet}}} q_2 \xrightarrow{\text{oracle resolves}} q_3 \xrightarrow{\text{grace period}} \text{paid}$$

Two paths lead out of q_0 back to the terminal deleted state q_{-1} , both returning the seed liquidity to the creator (the anti-spam creation fee, already paid to the DAO fund at creation, is *not* refunded): an explicit rejection $q_0 \xrightarrow{\text{oracle rejects}} q_{-1}$, and a *timeout* $q_0 \xrightarrow{t \geq t_0 + \tau_{\text{accept}}} q_{-1}$ triggered

by the per-block cron when the oracle neither accepts nor rejects within the governance-defined acceptance window τ_{accept} (default one hour). The timeout bounds how long an unresponsive oracle can hold a creator's seed capital hostage.

4. Pricing Mechanisms

4.1. Binary Markets: Constant Product Market Maker—For binary markets ($N = 2$), the protocol maintains two reserves R_A and R_B subject to the invariant:

$$k = R_A \cdot R_B$$

Initialization. Given seed liquidity L :

$$R_A = \lfloor L/2 \rfloor, \quad R_B = L - R_A, \quad k = R_A \cdot R_B$$

Bet placement. When bettor b_i wagers amount a on outcome A :

$$\begin{aligned} R'_A &= R_A + a \\ R'_B &= \lfloor k/R'_A \rfloor \\ w_i &= R_B - R'_B \quad (\text{tokens received}) \end{aligned}$$

Here w_i is the bettor's *weight*. It is a **computed** quantity — the number of outcome tokens minted by the curve — and is in general **not** equal to the staked amount a : depending on the current reserves, w_i may be larger or smaller than a . The weight is used solely to apportion the winners' pool at settlement (Section 5.1); it is never a currency-denominated claim.

Implied probability. The market-implied probability for each outcome is:

$$P(A) = \frac{R_A}{R_A + R_B}, \quad P(B) = \frac{R_B}{R_A + R_B}$$

Note that $P(A) + P(B) = 1$ by construction, eliminating the need for any external arbitrage mechanism to enforce price coherence.

4.2. Multi-Outcome Markets: LMSR—For markets with $N > 2$ outcomes, the protocol uses the Logarithmic Market Scoring Rule with softmax pricing.

Cost function:

$$C(\mathbf{q}) = b \cdot \ln \left(\sum_{j=1}^N \exp(q_j/b) \right)$$

where $\mathbf{q} = (q_1, \dots, q_N)$ are quantity parameters and b is the liquidity parameter.

Price function (softmax):

$$p_i(\mathbf{q}) = \frac{\exp(q_i/b)}{\sum_{j=1}^N \exp(q_j/b)}$$

By the definition of softmax, $\sum_{i=1}^N p_i = 1$ identically.

Cost of a trade. The cost to purchase Δ tokens on outcome i :

$$\text{cost}(\Delta, i) = C(\mathbf{q} + \Delta \cdot \mathbf{e}_i) - C(\mathbf{q})$$

Liquidity parameter. The parameter b is funded by the LP subsidy S :

$$b = \frac{S}{\ln N}$$

Numerical stability. The log-sum-exp computation uses the standard identity:

$$\ln \left(\sum_j \exp(x_j) \right) = \max(\mathbf{x}) + \ln \left(\sum_j \exp(x_j - \max(\mathbf{x})) \right)$$

5. Parimutuel Settlement and LP Guarantee

5.1. Unified Settlement Model—Both market types share an identical settlement mechanism. At resolution, the oracle declares winning outcome $O^* \in \mathbf{O}$. Let $\mathcal{W}$ be the set of bets on O^* and $\mathcal{L}$ be all other bets.

$$S_{\text{lose}} = \sum_{b_i \in \mathcal{L}} a_i$$

where a_i is the bet amount. The winners' pool W is computed as in Section 3.3. Each winning bettor $b_i \in \mathcal{W}$ receives:

$$\pi_i = a_i + \frac{w_i}{\sum_{b_j \in \mathcal{W}} w_j} \cdot W - \tau_i$$

where w_i is the bettor's weight (tokens from CPMM or LMSR) and τ_i is the time penalty on profit (Section 5.3).

5.2. Theorem: LP Principal Guarantee—

Theorem 1 (LP Principal Guarantee). *Under parimutuel settlement, the total money distributed equals the total money received, with LP principal L returned unconditionally. The LP principal is never at risk from betting outcomes.*

Proof. Consider the full money flow through a market:

$$\text{Money}_{\text{IN}} = L + \sum_{b_i \in \mathcal{W}} a_i + \sum_{b_i \in \mathcal{L}} a_i = L + \sum_{\text{all bets}} a_i$$

where the first sum $\sum_{b_i \in \mathcal{W}} a_i$ is the aggregate principal staked by *winners* (returned to them in full at settlement) and $S_{\text{lose}} = \sum_{b_i \in \mathcal{L}} a_i$ is the aggregate stake forfeited by *losers* (the sole source of the winners' pool and all fees). At resolution, the outgoing flows are:

$$\text{Money}_{\text{OUT}} = L + \sum_{b_i \in \mathcal{W}} a_i + W + f_{\text{oracle}} + f_{\text{creator}} + f_{\text{liq}}$$

Substituting $W = S_{\text{lose}} - f_{\text{oracle}} - f_{\text{creator}} - f_{\text{liq}}$:

$$\begin{aligned}
\text{Money}_{\text{OUT}} &= L + \sum_{b_i \in \mathcal{W}} a_i + S_{\text{lose}} \\
&= L + \sum_{b_i \in \mathcal{W}} a_i + \sum_{b_i \in \mathcal{L}} a_i = L + \sum_{\text{all bets}} a_i = \text{Money}_{\text{IN}}
\end{aligned}$$

■

Corollary 1. *The LP principal guarantee holds regardless of the weight assignment mechanism, the number of bettors, the distribution of bets across outcomes, or the fee parameters. It is a property of the settlement architecture, not the pricing curve.*

Corollary 2. *The maximum total payout to winners is S_{lose} (the entire losing pool), regardless of weights assigned by the AMM or LMSR. Therefore, winner payouts can never draw from LP capital.*

Remark (integer arithmetic and solvency under rounding). Theorem 1 is stated over the rationals, but the protocol operates on integers (milli-VIZ), and every division applies the floor operator $\lfloor \cdot \rfloor$. We show the guarantee survives discretization. Each fee and each winner share is rounded *down*:

$$f_{\bullet} = \lfloor S_{\text{lose}} \cdot \theta_{\bullet} / 10000 \rfloor, \quad \hat{\pi}_i^{\text{profit}} = \left\lfloor \frac{w_i}{\sum_j w_j} \cdot W \right\rfloor$$

Since $\lfloor x \rfloor \leq x$, the realized winners' distribution satisfies $\sum_{i \in \mathcal{W}} \hat{\pi}_i^{\text{profit}} \leq W$, and likewise $\sum_{\bullet} f_{\bullet} \leq S_{\text{lose}} \cdot (\theta_{\text{oracle}} + \theta_{\text{creator}} + \theta_{\text{liq}}) / 10000 \leq S_{\text{lose}}$. Therefore the realized outflow obeys

$$\text{Money}_{\text{OUT}}^{\text{realized}} = L + \sum_{i \in \mathcal{W}} a_i + \sum_{i \in \mathcal{W}} \hat{\pi}_i^{\text{profit}} + \sum_{\bullet} f_{\bullet} \leq L + \sum_{\text{all bets}} a_i = \text{Money}_{\text{IN}}$$

so the system is never insolvent. The non-negative residual

$$\varepsilon = S_{\text{lose}} - \sum_{i \in \mathcal{W}} \hat{\pi}_i^{\text{profit}} - \sum_{\bullet} f_{\bullet} \geq 0$$

is bounded by $\varepsilon < |\mathcal{W}| + 3$ milli-VIZ (one sub-unit per floored quantity) and is swept to the DAO fund as dust. Thus rounding can only *under*-distribute, never over-distribute: LP principal L remains exactly conserved and the inequality $\text{Money}_{\text{OUT}} \leq \text{Money}_{\text{IN}}$ holds at the granularity of the ledger.

5.3. Time Penalty on Late Bets—To mitigate information asymmetry from bets placed near expiration (when outcome uncertainty is reduced), the protocol imposes a configurable time penalty applied *only to profit*, never to principal.

Let $\Delta t = t_{\text{bet}} - t_{\text{now}}$ be the time to expiration and ω be the penalty window. If $\Delta t < \omega$:

$$r = 1 - \frac{\Delta t}{\omega}$$

$$\tau_{\text{rate}} = \begin{cases} r & \text{linear} \\ r^2 & \text{quadratic (default)} \end{cases}$$

$$\tau_i = \lfloor \pi_{\text{profit}, i} \cdot \tau_{\text{rate}} \cdot \tau_{\text{max}} / 10^6 \rfloor$$

where $\pi_{\text{profit},i}$ is the bettor's profit share (excluding principal) and τ_{max} is the maximum penalty in micro-units.

Invariant: $\pi_i \geq a_i$ for all winning bettors, since the penalty is applied exclusively to the profit component.

5.4. *LP Time-Weighted Fee Distribution*—LP fee shares are distributed proportionally to the product of deposit amount and remaining time to expiration:

$$\omega_j = \text{amount}_j \cdot \max(1, t_{\text{bet}} - t_{\text{deposit},j})$$

$$f_{\text{share},j} = \left\lfloor \frac{f_{\text{pool}} \cdot \omega_j}{\sum_m \omega_m} \right\rfloor$$

This creates a strong incentive for early liquidity provision. In a market of duration T , an LP depositing at time $t = 0$ earns T times more per unit capital than one depositing at $t = T - 1$.

6. “Lazy” Liquidity Pool

6.1. *Motivation*—Individual LP provision requires active market evaluation and selection. For retail participants, this creates an impractical knowledge and effort barrier. The “Lazy” Pool (so named because it requires no active market selection from depositors) solves this by accepting deposits and automatically allocating capital to activated markets.

6.2. *Pool Mechanics*—The Lazy Pool $\mathcal{L}_P$ is a singleton contract maintaining:

- B_{free} : unallocated balance
- B_{alloc} : capital deployed to active markets
- B_{earned} : realized profits
- S_{total} : total outstanding shares
- ρ : cumulative reward-per-share accumulator

Deposit. When user u deposits amount a :

$$\text{shares}_u = \begin{cases} a & \text{if } S_{\text{total}} = 0 \\ a \cdot S_{\text{total}} / B_{\text{free}} & \text{otherwise} \end{cases}$$

The deposit is locked for a governance-defined period t_{lock} .

Auto-allocation. On market activation ($q_0 \rightarrow q_1$):

$$a_{\text{alloc}} = B_{\text{free}} \cdot \alpha / 100$$

subject to $a_{\text{alloc}} \geq a_{\text{min}}$ and $B_{\text{alloc}} + a_{\text{alloc}} \leq B_{\text{total}} \cdot \alpha_{\text{max}} / 100$, where α is the per-market allocation percentage and α_{max} is the maximum total allocation cap.

Allocation is further gated by a **reward floor**: the pool provides capital to a market only if its LP fee f_{liq} meets a governance minimum $f_{\text{liq}}^{\text{min}}$ (default 2%). A market that offers LPs less than this earns nothing worth the pool's opportunity cost, so the pool allocates zero and the market's only depth is the creator's own seed. This makes the LP fee an explicit price the market pays for automated pool liquidity, and prevents the pool from subsidizing markets configured to route all fee revenue away from liquidity providers.

The allocation formula includes oracle quality adjustments:

$$a_{\text{alloc}} = B_{\text{free}} \cdot \frac{\alpha}{100} \cdot (1 - \beta)^{n_o} \cdot (1 - \gamma)^{f_o}$$

where n_o is the oracle’s active market count, f_o is the oracle’s active fault stamp count, and β, γ are governance-defined decay factors.

6.3. Reward Distribution (Lazy Accounting)—Rewards are distributed using a single global accumulator ρ (reward-per-share), following the MasterChef pattern (SushiSwap, 2020; Leshner & Hayes, 2019):

When a market resolves and the pool’s LP position returns profit π_{pool} :

$$\rho \leftarrow \rho + \frac{\pi_{\text{pool}} \cdot \text{PRECISION}}{S_{\text{total}}}$$

Any user u ’s pending reward is computable in $O(1)$:

$$\text{reward}_u = \text{pending}_u + \frac{\text{shares}_u \cdot (\rho - \rho_{\text{snapshot},u})}{\text{PRECISION}}$$

User records are updated only when that user acts (deposit or withdraw), giving $O(1)$ amortized cost regardless of participant count.

6.4. Opportunity-Cost Protection—The auto-allocation mechanism creates an attack vector: a malicious oracle could create long-duration zero-volume markets to lock pool capital. Three graduated defense mechanisms address this:

Graduated Recall. The market duration is divided into n_{steps} intervals. At each checkpoint k , if cumulative volume V_k satisfies:

$$V_k < a_{\text{alloc}} \cdot \theta_{\text{recall}} / 100$$

then a fraction δ_{recall} of the current allocation is recalled to the pool. After n consecutive low-volume checkpoints, the retained allocation is:

$$a_{\text{retained}} = a_{\text{alloc}} \cdot (1 - \delta_{\text{recall}})^n$$

For the default parameters ($\delta_{\text{recall}} = 10\%$, $n_{\text{steps}} = 10$), a completely idle market retains $(0.9)^{10} \approx 35\%$ of its original allocation.

Active Market Penalty. Each active market from oracle o multiplicatively reduces new allocations by factor $(1 - \beta)$. With $\beta = 5\%$ and 10 active markets, new allocations are $(0.95)^{10} \approx 60\%$ of the base rate.

Fault Stamps. Bad outcomes (no-contest, missed deadlines, dispute losses, zero-volume resolutions) generate penalty stamps on the oracle that further reduce allocations by factor $(1 - \gamma)^{f_o}$. Stamps auto-expire after a governance-defined clean-operation window.

6.5. Emergency Withdrawal—Users may exit the pool before lock expiration, subject to a penalty on *locked profit only*:

$$\text{penalty} = \max(0, V_{\text{total}} - P_{\text{deposited}}) \cdot \frac{S_{\text{locked}}}{S_{\text{total}}} \cdot \theta_{\text{emergency}} / 100$$

where V_{total} is the user’s total value (shares + pending rewards), $P_{\text{deposited}}$ is cumulative principal deposited, and $S_{\text{locked}}/S_{\text{total}}$ is the fraction of shares still locked. The penalty is redistributed to remaining pool participants via ρ .

6.6. *Opt-In Leverage Funded by the Pool*—The Lazy Pool plays a second, optional role from the same B_{free} : it funds a leverage subsystem. This subsystem is **opt-in, default-off, and governed by a median kill-switch**. We treat it separately from the rest of the paper for an important reason. **It is the one place where pool capital takes on credit risk, so it is *not* covered by the unconditional guarantee of Theorem 1.** Theorem 1 concerns the pool acting as a *market liquidity provider*, where principal is structurally insulated from betting outcomes. Leverage instead has the pool act as a *lender*, and lending carries risk that we bound rather than eliminate.

Position opening. A bettor posts collateral m . The pool lends margin λm at leverage factor λ from B_{free} , subject to caps on per-position size, pool fund fraction, and minimum market liquidity. No token is minted: the total stake $(1 + \lambda)m$ enters the market exactly as an ordinary bet would. The position’s curve weight, its collateral, and the lent amount are recorded on a dedicated position object. The pool’s obligation on the position is

$$\Omega = \lambda m(1 + r),$$

where r is the accrued interest. The loan plus interest is what the pool seeks to recover.

Liquidation against the recorded reserve snapshot. The hard case in leveraged binary markets is *jump risk*: a discontinuous price move can leave a naive “sell at the current price” liquidation unable to cover the loan. Onix does not liquidate at the current price. It closes the position via reverse-CPMM against the **reserve state recorded for that position**, so the position’s own market impact is unwound first rather than being paid for twice. Two paths drive recovery:

- (1) **Opposing-bet cascade** (`pm_place_bet`, liquidation reason 0) — an incoming opposing bet triggers a forced close of the leveraged position against its recorded snapshot.
- (2) **Settlement force-close** (`pm_leverage_resolve`) — at resolution the position is closed and the loan repaid from its weight before any profit accrues to the borrower.

On these paths the design target is full loan recovery, $\text{recovered} = \min(V_{\text{close}}, \Omega) \geq \lambda m$, with the realized loan-and-interest returning to the pool. We state this as a **design property, not a theorem**: the leverage settlement math is implemented and exercised in `consensus_sim`, but a closed-form proof of recovery under all reserve trajectories is left to future work (Section 13.2).

Where the borrower’s upside now goes. Because leverage closes against the curve rather than as a parimutuel share, its exit is governed by the early-exit deferred-claim mechanism (Section 7): the pool takes its obligation Ω from the close value first, and any residual $\max(V_{\text{close}} - \Omega, 0)$ becomes an *outcome-contingent deferred claim* rather than an immediate payout. Leverage is thus a **leveraged directional bet**, not a volatility harvest — the borrower profits only if the chosen outcome wins and the capped bucket has room. This is precisely what closes the historical uncovered-shortfall path (Section 7); with it, the borrower’s upside can never draw on LP principal.

Kill-switch decoupled from protection. The kill-switch disables *new* openings only. The liquidation paths are deliberately **not** gated by it, so disabling leverage never strips protection from already-open positions.

Risk isolation and reward. Leverage lending is confined to a governed fraction of B_{free} (`leverage_fund_percent`), so worst-case pool exposure is capped at the subsystem level and

cannot reach the market-LP principal that Theorem 1 protects. On the same-side voluntary-cancel path a borrower closing into an adverse move can still leave the pool short of Ω ; this shortfall is **bounded by the borrower’s collateral m** , which the protocol holds as first-loss margin, and cannot exceed it. Interest earned accrues to the pool through the same ρ accumulator (Section 6.3). Pool depositors therefore earn from two sources — losers’-pool fee shares (risk-free, Theorem 1) and leverage interest (bounded credit risk, opt-in) — and the two are accounted identically but governed independently.

6.7. *Design Decision: Real Depth Only (No Virtual/Phantom Liquidity)*—A natural proposal is to seed a market’s pricing curve with *virtual* (phantom) liquidity — a reserve offset ϕ added to flatten price impact but backed by no real capital and deleted at settlement, optionally tuned by governance. In the closed bet→cancel→settle loop this is value-conservative (it is the virtual-AMM technique), and it is tempting as a cold-start “stabilizer” for new, thin markets. Onix **deliberately does not implement it**. The same cold-start benefit is already delivered by Lazy-Pool auto-allocation (Section 6.2) — but with *real* capital, which additionally earns fees, has an accountable owner, and follows demand per market. We reject phantom depth because, applied carelessly, it damages the two things the protocol exists to protect — market structure and trust:

- (1) **Forgeable depth erodes trust.** Depth is meaningful as a signal only because it is costly real capital at risk. Free virtual depth makes “this market is deep and liquid” a forgeable claim — a thin or manipulated market can be dressed to look deep. Real capital makes the claim unforgeable.
- (2) **It silently distorts information aggregation.** Virtual depth flattens the weight curve, weakening the reward for early, correct information and making the displayed price unresponsive to news (“stable because unmovable” = a stale forecast). The right magnitude is per-market and volume-dependent; a single governance constant cannot track it and, set too high, degrades the very price-discovery property a prediction market provides.
- (3) **It is solvent only while never redeemed or used as collateral.** The moment depth backs a real outflow — cancellations, early withdrawals, leverage loans, shared/cross-market pools — the virtual part must be excluded everywhere or it leaks real money (e.g. a leverage loan sized or recovered against fake depth becomes real bad debt to pool depositors). Every “size against liquidity / pay the LP” branch becomes a footgun requiring “. . . but not the phantom part.” Real capital removes this entire class of excludability bugs by construction.
- (4) **It has no owner, no yield, no accountability.** Virtual depth bears no risk and earns no fee for anyone real; it is a service nobody is paid for and nobody answers for. For the markets it touches, it deletes the retail safe-yield product that is the protocol’s core hypothesis (Q3, H1).

The Lazy Pool is the same idea done with real numbers: auto-allocation smooths the launch of new markets, but the capital is redeemable, leverage-safe, fee-earning, owned by depositors, and self-correcting per market via graduated recall (Section 6.4). Onix therefore keeps **only real numbers** — every unit of depth is real capital that can be withdrawn, earns, and is accountable. This is a conscious tradeoff: we forgo a cheap virtual stabilizer to preserve the integrity of the price signal and the solvency of every real-money path.

7. Early Exit via Deferred Outcome-Contingent Claims

7.1. The Curve-Exit / Parimutuel Tension—The Onix market is a hybrid: a CPMM (binary) / LMSR (multi) **curve** prices both entry and early exit, while positions *held to resolution* settle parimutuel (Section 5). This creates a structural tension. Any round-trip through the curve — buy, then sell before settlement — realizes a trading P&L against the curve’s depth, which is the LPs, exactly like Uniswap impermanent loss. But Theorem 1 promises LPs *principal protection* with fee-only upside. The two properties are in direct conflict on any early-exit path.

Two consensus paths exit against the curve on a binary market:

- **Leverage** (`liquidate_position / pm_leverage_close`) — always, and force-closed at settlement, amplified by the pool loan (Section 6.6).
- **Regular bet cancel** (`cancel_bet`, a curve-priced refund) — during the betting window.

In the naive design both paths route a signed residual $\text{residual} = \text{stake} - \text{curve_refund}$ to the market’s `forfeit_pool`. When an early exit is *profitable* ($\text{curve_refund} > \text{stake}$), the residual is negative and `forfeit_pool` turns negative. At settlement the winners’ pool is

$$\text{winners_pool} = S_{\text{lose}} - \text{fees} + \text{forfeit_pool},$$

so if early-exit (especially leveraged) profit outran the losing stakes, $\text{winners_pool} < 0$. Floored to zero, the shortfall — call it *uncovered* — is charged to LP principal, or, once LP principal is exhausted, effectively *minted*. This is not hypothetical: it is reachable under a gate-respecting CPMM simulation (a one-sided pump produces $\text{uncovered} = 7316$ milli-VIZ), and an adversarial replay corpus hit it in 1163/1988 paired trajectories.

The root cause is precise: **a curve-priced exit pays a bonding-curve value that is not bounded by the losing pool**, whereas settlement pays parimutuel (bounded by S_{lose}). The unbounded gap lands on the LP — breaking the very guarantee the architecture exists to provide.

7.2. The Deferred-Claim Mechanism—Onix resolves the tension by making early exits *not* extract curve value from LPs at all. Instead, an exit records an **outcome-contingent deferred claim**, funded at settlement from a *bounded slice of the losing pool*.

Recorded on exit. A deferred-claim object stores (`position_id`, $\text{kind} \in \{\text{bet}, \text{leverage}\}$, `chosen_outcome`, `claim_a`

Regular bet cancel. Principal is returned *immediately and unconditionally*,

$$\text{refund} = \min(\text{curve_refund}, \text{stake}),$$

because it is the bettor’s own money. A cancel can cut losses or break even but never realizes curve-profit at cancel time. The profit tail

$$\text{claim_amount} = \max(\text{curve_refund} - \text{stake}, 0)$$

becomes a deferred claim on the chosen outcome. Crucially, only a *non-negative* residual $\text{stake} - \text{refund} \geq 0$ is ever routed to `forfeit_pool`; the negative-residual path that produced *uncovered* is eliminated.

Leverage close / liquidate. Collateral is *not* separately returned — it is first-loss margin for the pool. The pool recovers its obligation $\Omega = \lambda m(1 + r)$ from the close value V_{close} first; if $V_{\text{close}} < \Omega$, the collateral covers the gap. The residual

$$\text{claim_amount} = \max(V_{\text{close}} - \Omega, 0)$$

becomes a deferred claim on the chosen outcome. Two distinct predicates now govern a leveraged position, and they must not be conflated: *solvency* ($V_{\text{close}} \geq \Omega$, which governs loan recovery to the pool) versus *outcome-win* (which governs the right to a claim). A position can be solvent yet on the losing outcome — the pool is made whole and the claim is zero.

7.3. Settlement Distribution—At settlement the deferred claims are funded from a capped bucket:

- (1) Compute the bucket, clamped to the settlement headroom H (Section 7.4) so it can never exceed the pool available to winners:

$$\text{bucket} = \min\left(\left\lfloor \frac{\text{pm_early_exit_reward_cap_percent} \cdot S_{\text{lose}}}{10000} \right\rfloor, H\right)$$

(default cap = 3300 bp = 33% of the losing pool; the clamp binds only when fees are large enough that the cap would otherwise over-draw the pool).

- (2) Collect deferred claims on the **winning outcome only**; losing-outcome claims pay zero.
- (3) Pay them **FIFO by exit time** (first out, first paid) until the bucket is drained. There is no per-position cap — FIFO ordering plus the total bucket *is* the bound. A claim the remaining bucket cannot fully fund is paid partially; the unfunded remainder is a haircut.
- (4) **Any unused bucket returns to the winners' pool**, where held winning bets share it parimutuel.

7.4. Theorem: Bounded Early Exit, No Uncovered Shortfall—

Theorem 2 (Bounded Early Exit). *Let $c = \text{pm_early_exit_reward_cap_percent}/10000 \in [0, 1)$, and let the funding bucket be clamped to the settlement headroom:*

$$\text{bucket} = \min(\lfloor c S_{\text{lose}} \rfloor, H), \quad H = \max(0, S_{\text{lose}} - \text{fees} - f_{\text{fixed}}^{\text{paid}} + \text{honest_forfeits}),$$

where H is the winners' pool available immediately before claims are paid and $\text{honest_forfeits} \geq 0$. Then, for any market and any valid fee parameters, the paid early-exit claims never exceed the bucket, the winners' pool is non-negative, and LP principal is never touched by early exits:

$$\text{paid_claims} \leq \text{bucket} \leq H, \quad \text{winners_pool} = H - \text{paid_claims} \geq 0.$$

Proof. The FIFO distribution (Section 7.3) stops paying once cumulative payments reach the bucket, so $\text{paid_claims} \leq \text{bucket}$. By the clamp, $\text{bucket} \leq H$, hence $\text{paid_claims} \leq H$. The winners' pool is exactly H minus what was paid to early exiters, so $\text{winners_pool} = H - \text{paid_claims} \geq 0$. No shortfall is charged to LP principal, and because outflow never exceeds $S_{\text{lose}} + L + \sum_{i \in \mathcal{W}} a_i$, no tokens are minted. After the mechanism of Section 7.2 only non-negative residuals reach forfeit_pool , so $\text{honest_forfeits} \geq 0$ and H is well defined. ■

The clamp is what makes the guarantee *unconditional*. Without it, one would need the governance invariant $\theta_{\text{total}}/10000 + c \leq 1$ (with $\theta_{\text{total}} = \theta_{\text{oracle}} + \theta_{\text{creator}} + \theta_{\text{liq}}$), giving the intuitive bound $\text{winners_pool} \geq (1 - c) S_{\text{lose}} - \text{fees} \geq 0$. That invariant is *not* enforced by parameter validation — per-market fees are bounded only by $\theta_{\text{total}} \leq 100\%$ at market creation, with no chain-level cap on the creator or liquidity fee — so a valid market with high fees combined with a non-zero cap could otherwise drive winners_pool negative and charge the shortfall to LP principal. In the default regime (fees $\approx 2\%$, cap 33%) the min is inactive and $\text{bucket} = \lfloor c S_{\text{lose}} \rfloor$, recovering the intended cap exactly; the clamp binds only in the adversarial high-fee corner, where it preserves solvency at the cost of a deeper early-exit haircut.

Corollary 3 (No-mint, LP-safe, loser-never-profits). *Under Theorem 2: (i) an uncovered shortfall is impossible by construction, so no minting occurs and LP principal — including the Lazy Pool’s market-LP principal — is never touched by early exits or by leverage impermanent loss; (ii) a claim on a losing outcome pays zero, so a losing outcome never profits; and (iii) held winners receive at least $(1 - c)S_{lose}$ of the losing pool plus any unused bucket.*

7.5. Economic Interpretation: A Bounded Liquidity Discount—The deferred claim reframes early exit as a *choice* with clean incentives. A holder who stays to resolution receives a full parimutuel share of the losing pool. A holder who exits early instead receives (a) immediate return of principal (regular bet) or loan-satisfied collateral treatment (leverage), plus (b) a *capped, priority-ordered, outcome-contingent* claim of at most c of the losing pool, paid FIFO. Because the early-exit payoff is strictly a capped-and-contingent version of the hold payoff, **there is no arbitrage against holding**: one cannot exit early to extract *more* than holding would have yielded. The discount is deliberate — it is the price of liquidity and of transferring outcome risk earlier — and it is exactly what makes the LP guarantee survive a curve that also prices exits.

This also gives leverage a coherent economic identity. Under the deferred-claim rule, borrowing from the Lazy Pool to open a position is a leveraged *directional* bet: the borrower amplifies exposure to a chosen outcome, the pool is repaid its loan and interest first from the curve close value, and the borrower’s upside is a bounded contingent claim that competes FIFO with other early exiters for the capped bucket. The volatility-harvest interpretation — open, ride the curve, cash out the swing at LP expense — is structurally removed.

7.6. Governance Parameter and Implementation—The cap is a median-voted validator parameter `pm_early_exit_reward_cap_percent` (uint16, basis points, default 3300), added to `chain_properties.pm` with `validate()` bounding it to ≤ 10000 . Deferred claims live in a dedicated `pm_deferred_claim_object` (indexed by market and by exit order), included in the snapshot allowlist with an import handler. A virtual operation `pm_early_exit_claim_paid(account, market, kind, c)` is emitted in the settlement distribution loop — exposing any bucket-exhaustion haircut via the gap between claimed and paid — and routed to the exiter’s account history (a bare balance adjustment would leave no trace). A read API `get_deferred_claims` returns the FIFO queue, empty on a settled market once claims are consumed.

Three defensive measures harden the mechanism against adversarial parameter extremes and denial-of-service. First and most important, the funding bucket is **clamped to the settlement headroom H** (Section 7.4): the amount actually drawn for claims is $\min(\lfloor cS_{lose} \rfloor, H)$, which is what makes Theorem 2 unconditional rather than contingent on a $\text{fees} + \text{cap} \leq 100\%$ governance invariant — the latter is not enforced by parameter validation, since the creator and liquidity fees have no chain-level cap. Second, the per-market count of deferred-claim objects is capped by `MAX_PM_DEFERRED_CLAIMS_PER_MARKET`; beyond the cap an early exit skips claim creation and the tail simply stays in the curve paying zero — it is deliberately *not* routed to `forfeit_pool`, which would double-count and thereby mint. Third, an always-on diagnostic fires if uncovered > 0 ever occurs (unreachable once the bucket is clamped), and a $\lfloor \cdot \rfloor_{\geq 0}$ floor on the winners’ pool remains as a final backstop. These are belt-and-suspenders: the clamp gives the guarantee by construction, and the diagnostic makes any latent violation loud rather than silent.

Verification. The conservation identity of Theorem 2 was validated in a settlement model across adversarial scenarios (one-sided pumps, thin losing sides, bucket exhaustion, fee extremes,

and $\Omega > \text{total stake from long-lived leverage}$) and in the `consensus_sim` harness. It was further confirmed end-to-end on the VIZ testnet: a cancel recorded a deferred claim; at settlement the virtual operation reported the claim paid at the capped bucket with a visible haircut (`claimed > paid`), and the claim was consumed — the full lifecycle behaving exactly as the model predicts.

8. Dispute Resolution Under DAO Governance

8.1. Bonded Oracle Model—Each oracle o must maintain an insurance bond $I_o \geq I_{\min}$. The bond creates accountability: oracles who misresolve markets, miss deadlines, or lose disputes have their insurance slashed. Oracle revenue derives from:

- (1) A fixed per-market fee f_{fixed} (compensating insurance staking)
- (2) A percentage fee f_{oracle} from the losers' pool at resolution

These two fees enter the system at different points and must not be conflated. The **percentage fee** f_{oracle} is deducted from S_{lose} at resolution and appears in the money flow of Theorem 1 (Section 5.2). The **fixed fee** f_{fixed} is a direct creator→oracle transfer executed at market *acceptance*, before any betting; it is therefore *not* part of the betting/LP money flow — it neither adds to the seed liquidity L nor draws from S_{lose} , and so does not appear in Theorem 1. (It is omitted from the fee vector θ of Section 3.3 for the same reason: θ collects only the resolution-time, losers-funded percentage fees.) For self-oracle markets no transfer occurs and $f_{\text{fixed}} = 0$.

The oracle acceptance flow implements an offer-quote mechanism: the creator publishes fee ceilings, and the oracle freezes its actual terms (bounded by both the creator ceiling and a governance cap) at acceptance. Acceptance is time-bounded by the window τ_{accept} (Section 3.4): a market left pending beyond it is auto-expired by the per-block cron, which refunds the creator's seed liquidity while retaining the anti-spam creation fee — so an unresponsive oracle cannot indefinitely freeze the seed.

8.2. Two-Mode Dispute System—Any bettor may challenge a resolution within a grace period Δt_{grace} by escrowing a dispute fee d . During disputes, all payouts are frozen. The protocol supports two dispute modes per market:

Committee mode ($\delta_{\text{mode}} = 0$). The entire token-holder electorate resolves the dispute via stake-weighted public vote. Each voter v 's weight is:

$$w_v = s_v + \text{shares}_{v,\text{pool}} \cdot \text{NAV}_{\text{pool}} / S_{\text{total}}$$

where s_v is the voter's vesting shares and the second term converts Lazy Pool stake into equivalent governance weight. Votes are public and revisable until the voting period closes — this is a deliberate design choice: disputes are *transparent public hearings*, not secret ballots. New evidence can change votes.

This transparency carries a known cost. Because the running tally is visible, a voter can defer until late in the window and condition its ballot on others' revealed positions (a last-mover advantage). The protocol accepts this tradeoff on purpose: for a DAO, the auditability and legitimacy of an open hearing are judged more valuable than ballot secrecy, and a flipped outcome still requires moving a stake-weighted majority. Manipulation is deterred not by secrecy but by (i) the stake-weighted approval threshold θ_{approve} , (ii) a multi-day voting window that dilutes any single timing edge, and (iii) the resolver-reward alignment analyzed in Section 8.4. A commit-reveal ballot was explicitly considered and **rejected** for this reason — concealing votes

would defeat the public-hearing property that gives the verdict its legitimacy. We revisit this as an honest limitation in Section 13.2.

The dispute is upheld if the approval percentage exceeds a governance threshold θ_{approve} :

$$\frac{\sum_{v:\text{vote}_v=\text{approve}} w_v}{\sum_v w_v} \geq \frac{\theta_{\text{approve}}}{10000}$$

Account mode ($\delta_{\text{mode}} = 1$). A named dispute resolver (recommended: multisig) issues a binding verdict.

8.3. *Dispute Outcomes and Incentive Compatibility*—**Oracle wrong (dispute upheld)**:

$$\text{reward}_{\text{pool}} = \min(d \cdot \mu, I_o)$$

where μ is the reward multiplier. The reward pool is split:

- Disputer receives d (fee refund) + $\text{reward}_{\text{pool}}/\mu$
- Resolver/voters receive $\text{reward}_{\text{pool}} - \text{reward}_{\text{pool}}/\mu$
- Remaining insurance: additional penalty $\rightarrow$ DAO fund

Oracle right (dispute rejected):

$$d \rightarrow 50\% \text{ resolver, } 50\% \text{ oracle}$$

Oracle non-responsive (auto-close at 14 days):

- Disputer: fee refunded
- Oracle: d slashed from insurance
- All bets and LP positions: full refund
- Slashed amount distributed proportionally to all participants

8.4. *Game-Theoretic Analysis*—**Oracle incentive compatibility**. The oracle's expected payoff from honest resolution vs. manipulation is:

$$\mathbb{E}[\pi_{\text{honest}}] = f_{\text{fixed}} + f_{\text{oracle}} \quad (\text{per market})$$

$$\mathbb{E}[\pi_{\text{dishonest}}] = p_{\text{detect}} \cdot (-I_o \cdot \theta_{\text{penalty}} - B_{\text{ban}}) + (1 - p_{\text{detect}}) \cdot g_{\text{manipulation}}$$

where p_{detect} is the probability of dispute, θ_{penalty} is the insurance slash fraction, B_{ban} is the present value of a ban (lost future revenue), and $g_{\text{manipulation}}$ is the one-time manipulation gain.

Honest behavior is a Nash equilibrium when:

$$f_{\text{fixed}} + f_{\text{oracle}} > (1 - p_{\text{detect}}) \cdot g_{\text{manipulation}} - p_{\text{detect}} \cdot (I_o \cdot \theta_{\text{penalty}} + B_{\text{ban}})$$

The protocol ensures this by requiring $I_o \gg g_{\text{manipulation}}$ and maintaining high p_{detect} through public dispute visibility and committee participation incentives.

Disputer incentive. A rational bettor files a dispute when the expected reward exceeds the dispute fee:

$$\mathbb{E}[\text{reward}_{\text{dispute}}] = p_{\text{upheld}} \cdot (d + d \cdot (\mu - 1)/\mu) > d$$

This simplifies to $p_{\text{upheld}} > 1/\mu$, establishing a natural threshold for dispute filing.

Committee voting incentive. In committee mode, voters participate because their Lazy Pool stake (and thus governance weight) directly earns a share of the resolver reward pool. The expected reward for voter v is:

$$\mathbb{E}[\pi_v] = \frac{w_v}{\sum w_j} \cdot \text{voter_reward_pool} \cdot p_{\text{upheld}}$$

8.5. *No-Contest Declaration*—An oracle unable to verify an outcome may declare no-contest at reduced cost (50% of the dispute penalty from insurance). This creates a three-tier incentive gradient:

Action	Oracle Cost	Ban Risk
Voluntary no-contest	$0.5 \cdot d$ from insurance	None
Dispute loss	$I_o \cdot \theta_{\text{penalty}} + \text{extra}$	Yes
Missed deadline	$I_o \cdot \theta_{\text{miss}}$	None

A no-contest declaration is itself disputable, with the resolver choosing from three outcomes (A wins, B wins, or confirm no-contest), preventing abuse.

9. The VIZ Token in the Prediction Market Experiment

9.1. *Token Utility*—The VIZ token serves four distinct functions within the Onix Protocol ecosystem:

- (1) **Medium of exchange.** All bets, LP deposits, oracle insurance, and dispute fees are denominated in VIZ. The protocol never mints tokens — it is strictly zero-sum at the consensus level.
- (2) **Governance weight.** VIZ staked as vesting shares (or deposited in the Lazy Pool) confers voting power in DAO committee disputes and chain parameter governance. This creates direct utility for token holding beyond speculation.
- (3) **Oracle collateral.** Oracle insurance bonds are denominated in VIZ. The bond must exceed the oracle’s potential manipulation profit, creating a demand for token accumulation by oracle operators.
- (4) **Dispute escrow.** Dispute fees are escrowed in VIZ, creating a cost for frivolous disputes and a reward mechanism for valid challenges.

9.2. *Token Demand Feedback Loop*—The protocol creates a structural demand cycle:

$$\begin{aligned}
 &\text{LP deposits} \xrightarrow{\text{lock}} \text{reduced circulating supply} \\
 &\text{Oracle insurance} \xrightarrow{\text{lock}} \text{reduced circulating supply} \\
 &\text{Active bets} \xrightarrow{\text{lock}} \text{reduced circulating supply} \\
 &\text{Governance weight} \xrightarrow{\text{utility}} \text{demand for staking}
 \end{aligned}$$

The total locked supply at any time is:

$$S_{\text{locked}} = S_{\text{pool}} + \sum_o I_o + \sum_{\text{active bets}} a_i + \sum_{\text{disputes}} d_k$$

where S_{pool} is the total withdrawal-locked Lazy-Pool balance (free *and* market-allocated portions are both locked for the deposit lock period), I_o is oracle insurance, a_i are active bet stakes, and d_k are escrowed dispute fees. This locked supply reduces available circulating supply, potentially creating upward price pressure as protocol usage grows — the same bet every protocol-native token makes, here explicitly tied to measurable utility.

9.3. Experimental Value of the Token—The VIZ token in this experiment serves as a **measurement instrument**:

- **Price as signal.** Token price movements in response to protocol events (market launches, high-volume resolutions, dispute outcomes) provide a continuous market-based assessment of the protocol’s perceived value.
- **Governance participation rate.** The fraction of token holders participating in dispute votes measures the viability of DAO-based arbitration.
- **Lazy Pool accumulation rate.** The rate of pool deposits measures retail demand for risk-free LP yield, directly testing hypothesis Q3.

10. Full Market Cycle: A Worked Example

We trace a complete binary market lifecycle to illustrate the protocol’s operation.

10.1. Setup—

- **Market:** “Will event X occur by date Y?” (Binary: Yes/No)
- **Oracle:** o with insurance $I_o = 5000$ VIZ
- **Seed liquidity:** $L = 200$ VIZ from creator
- **Fee parameters:** $\theta_{\text{oracle}} = 50$ bp, $\theta_{\text{creator}} = 50$ bp, $\theta_{\text{liq}} = 100$ bp
- **Lazy Pool:** $B_{\text{free}} = 10,000$ VIZ, $\alpha = 2\%$, allocates $a_{\text{alloc}} = 200$ VIZ

Remark (Units). Amounts are displayed in VIZ for readability, but the protocol stores and computes them as **integer milli-VIZ (mVIZ)**, with $1 \text{ VIZ} = 1000 \text{ mVIZ}$, and every division is floored (cf. the Theorem 1 remark). The fee lines below are shown in mVIZ so that $\lfloor \cdot \rfloor$ is exact; the VIZ equivalent is given alongside. This resolves the apparent rounding mismatch that arises if one floors over whole-VIZ quantities (e.g. $\lfloor 80 \times 50 / 10000 \rfloor$ is 0 in VIZ but $400 \text{ mVIZ} = 0.4 \text{ VIZ}$ in the real units).

10.2. Initialization—

$$R_A = 100, \quad R_B = 100, \quad k = 10,000$$

Lazy Pool auto-allocates 200 VIZ as additional LP:

$$R_A = 200, \quad R_B = 200, \quad k = 40,000$$

10.3. Betting Phase—

- **Alice** bets $a_1 = 50$ VIZ on Yes (side A):

$$R'_A = 250, \quad R'_B = \lfloor 40000 / 250 \rfloor = 160, \quad w_1 = 200 - 160 = 40$$

- **Bob** bets $a_2 = 80$ VIZ on No (side B):

$$R'_B = 240, \quad R'_A = \lfloor 40000 / 240 \rfloor = 166, \quad w_2 = 250 - 166 = 84$$

Implied probability: $P(\text{Yes}) = 166 / 406 \approx 41\%$, $P(\text{No}) = 240 / 406 \approx 59\%$.

10.4. *Resolution*—Oracle declares **Yes** wins. Alice is the sole winner; Bob forfeits 80 VIZ.

$$\begin{aligned}
S_{\text{lose}} &= 80,000 \text{ mVIZ (80 VIZ)} \\
f_{\text{oracle}} &= \lfloor 80,000 \times 50 / 10000 \rfloor = 400 \text{ mVIZ (0.4 VIZ)} \\
f_{\text{creator}} &= \lfloor 80,000 \times 50 / 10000 \rfloor = 400 \text{ mVIZ (0.4 VIZ)} \\
f_{\text{liq}} &= \lfloor 80,000 \times 100 / 10000 \rfloor = 800 \text{ mVIZ (0.8 VIZ)} \\
W &= 80,000 - 400 - 400 - 800 = 78,400 \text{ mVIZ (78.4 VIZ)}
\end{aligned}$$

Alice's payout (sole winner, $w_1/w_1 = 1$):

$$\pi_1 = 50,000 + \lfloor 78,400 \times 40 / 40 \rfloor - \tau_1 = 128,400 \text{ mVIZ} - \tau_1 \text{ (128.4 VIZ} - \tau_1)$$

LP return:

- Creator LP: 200 VIZ principal + time-weighted share of 0.8 VIZ fee pool
- Lazy Pool LP: 200 VIZ principal + time-weighted share of 0.8 VIZ fee pool

Verification (Theorem 1):

$$\begin{aligned}
\text{Money}_{\text{IN}} &= 200 + 200 + 50 + 80 = 530 \\
\text{Money}_{\text{OUT}} &= 200 + 200 + 50 + 78.4 + 0.4 + 0.4 + 0.8 = 530 \quad \checkmark
\end{aligned}$$

10.5. *Dispute Scenario*—If Bob disputes within $\Delta t_{\text{grace}} = 12\text{h}$, paying $d = 10$ VIZ:

- (1) All payouts freeze.
- (2) Oracle must respond within 12 hours.
- (3) In committee mode, all token holders vote (stake-weighted, public, revisable).
- (4) If dispute upheld: payouts recalculated with corrected outcome, oracle insurance slashed.
- (5) If dispute rejected: original payouts proceed, Bob loses 10 VIZ (split 50/50 to oracle and resolver).
- (6) If no resolution within 14 days: auto-close with full refunds.

10.6. *Early-Exit Scenario*—Suppose that instead of holding, Alice cancels her Yes bet mid-market after Bob's bet has moved the curve in her favor, so the curve refund exceeds her 50 VIZ stake. Under the deferred-claim mechanism (Section 7) she is returned her principal immediately, $\text{refund} = \min(\text{curve_refund}, 50) = 50$ VIZ, and the profit tail $\max(\text{curve_refund} - 50, 0)$ is recorded as a deferred claim on Yes rather than paid from the curve. At resolution, if Yes wins, her claim is paid FIFO from the capped bucket $\lfloor 0.33 \times S_{\text{lose}} \rfloor$; if that bucket is already drained by earlier exiters, she takes a haircut; if No wins, her claim pays zero. LP principal is untouched in every branch, and the winners who *held* to resolution receive at least $(1 - 0.33) S_{\text{lose}}$ of the losing pool plus any unused bucket — concretely illustrating Theorem 2.

11. Anti-MEV: Batch and Commit-Reveal Betting

11.1. *MEV in Prediction Markets*—In continuous-time prediction markets, front-running and sandwich attacks extract value from bettors. When a large bet is broadcast to the mempool, an attacker can:

- (1) Front-run: place a bet on the same side before the large bet, profiting from the price movement
- (2) Sandwich: place bets on both sides around the large bet

11.2. Batch Settlement—The protocol supports opt-in batch betting (binary markets). Bets submitted within an epoch of E blocks are queued and settled at a **uniform price** at the epoch boundary. Only the net residual (aggregate demand difference) moves the AMM:

$$\Delta R_A = \sum_{\text{batch}} a_{i,A} - \sum_{\text{batch}} a_{i,B}$$

This eliminates intra-batch ordering advantage: all bets in a batch receive identical pricing regardless of submission order.

11.3. Commit-Reveal—For stronger MEV protection, bettors may use a two-phase commit-reveal scheme:

- (1) **Commit:** Submit $H(\text{bet}||\text{nonce})$ with escrow. The bet’s direction and amount are hidden.
- (2) **Reveal:** After the epoch closes, submit $(\text{bet}, \text{nonce})$ to execute at the batch price.

Unrevealed commitments forfeit a governance-defined penalty percentage of the escrow, preventing spam commitments.

12. Implementation

12.1. Consensus-Level Operations on VIZ DLT—The protocol is implemented as first-class consensus-validated operations on the VIZ distributed ledger — not smart contracts, not custom payloads. VIZ DLT provides ~ 3 -second block times, Delegated Proof of Stake consensus, and named accounts (Graphene-style) with no general-purpose virtual machine.

Every financial action (market creation, bet placement, oracle resolution, dispute filing, LP deposit/withdrawal) is a `pm_*` operation validated by every validator node. Invalid operations are rejected before block inclusion.

Layer	Examples	Consensus-Validated
Protocol operations	<code>pm_create_market</code> , <code>pm_place_bet</code> , ...	Yes — every node validates
Virtual operations	<code>pm_payout</code> , <code>pm_early_exit_claim_paid</code> , ...	Yes — deterministic, block-time
Metadata	Market descriptions, dispute evidence	No — client-side only

12.2. On-Chain State—All protocol state resides in chainbase indexed objects. Key objects include:

- `pm_market_object`: market configuration, CPMM reserves, fee parameters, state
- `pm_bet_object`: bettor positions with weight and time penalty
- `pm_liquidity_object`: LP positions with time-weight
- `pm_lazy_pool_object`: singleton pool state (balances, shares, reward accumulator)
- `pm_oracle_object`: oracle registration, insurance, reputation counters
- `pm_dispute_object`: dispute state and resolution
- `pm_deferred_claim_object`: early-exit outcome-contingent claims (FIFO by exit time)

Reputation metrics are computed on read (not stored), ensuring consistency without additional write operations.

12.3. Governance Parameters—All economic parameters (fees, penalties, insurance requirements, dispute windows, lazy pool settings, and the early-exit reward cap `pm_early_exit_reward_cap_percent`) are delegate median-voted. Each elected validator publishes preferred values; the network computes the median. Parameters change without hard forks or deployments. Kill-switches allow governance to disable subsystems (leverage, commit-reveal) by median vote.

12.4. Scalability—Because every operation is consensus-validated, the protocol’s cost model matters at scale (thousands of concurrent markets and bets). The design keeps per-block and per-operation work bounded:

- **$O(1)$ user-facing operations.** Bet placement, cancellation, and LP deposit/withdraw touch a fixed number of indexed objects (the market, the bettor’s position, and — for the pool — the singleton accumulator). None iterates over all participants. Reward distribution uses the MasterChef accumulator ρ (Section 6.3), so a pool with n depositors settles rewards in $O(1)$ per actor rather than $O(n)$ per resolution.
- **Bounded deferred work per block.** Settlement is the only fan-out step (a market with m winning bets generates m `pm_payout` virtual operations; early-exit distribution adds at most one `pm_early_exit_claim_paid` per surviving deferred claim, itself capped per market). To prevent a single large resolution from bloating a block, payout and cron processing are rate-limited by the median parameter `pm_processing_cap_per_block`: at most a fixed number of payouts/cron items are processed per block, and the remainder carry to subsequent blocks. A round-robin cursor over markets ensures new markets are not starved by low-id incumbents. Worst-case per-block work is therefore $O(\text{cap})$, independent of how many markets resolve in the same interval.
- **No write amplification from reputation.** The 14 oracle metrics are counters updated only on the oracle’s own actions; the composite reliability score is computed **on read**, not written each block (Section 12.2). Market discovery metadata is built by a non-consensus plugin and never enters block validation.
- **State growth.** State is linear in live objects (markets, open positions, active disputes, unsettled deferred claims). Resolved markets and paid positions are terminal and prunable by clients; consensus retains only what open obligations require.

The binding constraint at very high market counts is thus block space for settlement fan-out, which `pm_processing_cap_per_block` converts from a latency spike into bounded, amortized throughput rather than a consensus-halting cost. A quantitative stress simulation across many concurrent markets of varying volume and volatility is part of the experimental program (Section 13.3, H4).

13. Discussion

13.1. Comparison with Existing Approaches—

Dimension	Onix	Polymarket	Std LMSR	Uniswap AMM
LP risk	Zero (structural)	Inventory risk	Up to $b \ln N$	Impermanent loss
LP knowledge	Low (deposit)	High (manage orders)	Medium	Medium-High
Pricing continuity	Continuous	Discrete (book)	Continuous	Continuous
Fee extraction	Losers only, at resolution	Bid-ask spread	Spread	Every trade
Oracle model	Bonded + DAO dispute	UMA Oracle	Operator	N/A
Price coherence	Math invariant	Arbitrage-dependent	Math invariant	Math invariant
CTF split/merge	No	Yes	No	No

13.2. Limitations and Honest Tradeoffs—

- (1) **LP yield is volume-dependent, not depth-dependent.** A market with high subsidy and low volume earns the same absolute fees as one with low subsidy and equal volume. The subsidy provides depth (lower slippage) but not yield.
- (2) **LP profit is not guaranteed.** If a market resolves with zero losing bets, no fees are generated. LP principal is returned, but yield may be zero.
- (3) **Parimutuel tokens are not fixed-value instruments.** Unlike standard LMSR where 1 winning token = 1 currency unit, Onix tokens are proportional claims on the losers' pool. If all bettors pick the winner, everyone breaks even.
- (4) **DPoS governance tradeoffs.** The delegate-voted parameter model has known concentration risks inherent to DPoS (shared with EOS, Hive, Tron). The Lazy Pool governance-weight mechanism partially mitigates this by broadening the effective electorate.
- (5) **Token liquidity dependency.** Economic guarantees (insurance bonds, dispute fees) scale with token market value. The protocol assumes utility drives demand — the standard assumption for protocol-native tokens.
- (6) **Leverage adds bounded credit risk to the pool.** The opt-in leverage subsystem (Section 6.6) is the one component *not* covered by Theorem 1. When enabled, the pool acts as a lender and can incur a shortfall on the same-side-cancel path, bounded by borrower collateral and isolated to a governed fund fraction. We claim full loan recovery on the cascade and settlement paths as a *design property verified in simulation*, not as a proven theorem; a closed-form recovery proof under arbitrary reserve trajectories is open work. The subsystem is default-off and disableable by median vote, so the unconditional LP guarantee can always be restored.
- (7) **Early exit is a bounded discount, and the cap is a policy choice.** The deferred-claim mechanism (Section 7) gives a *hard*, unconditional LP guarantee (Theorem 2, via the headroom clamp), but at the cost of making early exit a capped, FIFO-ordered, outcome-contingent payoff rather than a mark-to-curve cash-out. The cap c is a governance policy choice: too low a cap makes early exit unattractive and thins secondary liquidity, too high a cap erodes the reward to holders — and in the high-fee corner the settlement clamp further shrinks the effective bucket to preserve solvency, deepening the early-exit haircut. Finding the value that maximizes secondary liquidity without discouraging holding is an empirical question (Section 13.3, H8), not one the theory settles.
- (8) **Public dispute voting has a timing tradeoff.** Open, revisable committee ballots (Section 8.2) admit a last-mover advantage and were chosen deliberately over commit-reveal

for auditability. This is a value judgment (transparency over secrecy), not a proof that open voting is manipulation-optimal; deployments that prioritize secrecy would need account-mode resolution instead.

13.3. Experimental Hypotheses—The deployed protocol is designed to test the following hypotheses:

H1 (Liquidity flywheel). *Zero-risk LP provision attracts retail capital sufficient to produce market depth that meaningfully reduces slippage relative to comparable platforms.* Measurable: Lazy Pool total deposits, average market depth (reserve ratios), slippage per unit bet size.

H2 (Information aggregation). *AMM/LMSR pricing with parimutuel settlement produces probability estimates of comparable accuracy to CLOB-based prediction markets for equivalent information conditions.* Measurable: Brier scores, calibration curves, comparison with external benchmarks.

H3 (Oracle reliability under DAO governance). *Bonded oracles with committee-mode dispute resolution achieve resolution accuracy comparable to centralized oracle services, with dispute rates below a sustainable threshold.* Measurable: oracle dispute rate, dispute loss rate, average resolution time, reliability score distribution.

H4 (Lazy Pool sustainability). *The graduated recall and fault stamp mechanisms prevent oracle exploitation of the Lazy Pool, maintaining positive net yield for pool depositors across diverse market portfolios.* Measurable: pool NAV over time, recall frequency, fault stamp distribution, net yield per share.

H5 (Token demand correlation). *Protocol usage (volume, market count, LP deposits) positively correlates with token staking demand and governance participation rate.* Measurable: token staking rate, dispute vote participation, Lazy Pool deposit rate, correlation analysis.

H6 (Anti-MEV effectiveness). *Batch settlement and commit-reveal mechanisms reduce measurable MEV extraction compared to continuous instant betting.* Measurable: price impact asymmetry (batch vs. instant), sandwich attack frequency, bettor execution quality.

H7 (Leverage solvency). *Under live trajectories, snapshot-based liquidation recovers the loan on the cascade and settlement paths, confining realized pool shortfall to the same-side-cancel path and within borrower collateral, so that leverage interest is net-accretive to pool yield.* (This tests the design property of Section 6.6 empirically, in lieu of a closed-form recovery proof.) Measurable: realized loan-recovery ratio per liquidation event ($\text{recovered}/\lambda m$), frequency and magnitude of same-side-cancel shortfalls relative to posted collateral, leverage interest as a share of total pool yield, pool NAV with vs. without leverage enabled.

H8 (Early-exit liquidity discount). *The deferred-claim mechanism (Section 7) provides useful secondary liquidity — bettors and leveraged positions do exit early — while the capped, FIFO bucket keeps the early-exit payoff strictly below the hold payoff, so held winners are not diluted and no early exit ever draws on LP principal.* Measurable: early-exit volume and frequency, distribution of paid/claimed (haircut) ratios from the `pm_early_exit_claim_paid` virtual operation, realized early-exit yield vs. hold yield on matched positions, and the empirical uncovered rate (expected: identically zero) as a live check of Theorem 2.

14. Conclusion

We have presented the Onix Protocol, a prediction market architecture that achieves a structural LP principal guarantee by decoupling the pricing function (CPMM for binary, LMSR for multi-outcome) from the settlement function (parimutuel, losers-fund-winners). We proved that this guarantee holds for both market types regardless of weight assignment, fee parameters, or betting distribution (Theorem 1), and that it *survives the presence of a curve that also prices early exits* — the deferred outcome-contingent claim mechanism (Theorem 2) lets bettors and leveraged positions leave early without ever realizing a trading P&L against LP depth, with no uncovered shortfall and no minting.

The “Lazy” Pool mechanism enables passive retail LP participation with automated capital deployment and graduated opportunity-cost protection. The two-mode dispute system — combining bonded oracles with stake-weighted DAO committee voting — provides a viable alternative to centralized arbitration while maintaining oracle incentive compatibility.

The protocol is implemented as consensus-level operations on the VIZ distributed ledger and is designed as an experiment testing eight explicit hypotheses about liquidity bootstrapping, information aggregation accuracy, DAO governance viability, leverage solvency, early-exit liquidity, and token demand dynamics. The opt-in leverage subsystem is the single component that trades the unconditional LP guarantee for bounded, default-off pool credit risk; everything else preserves the structural guarantees of Theorems 1 and 2.

The mathematical properties are verifiable. The economic hypotheses will be tested by market participation.

Notes and References

¹ Hanson, R. (2003). Combinatorial Information Market Design. *Information Systems Frontiers*, 5(1), 107–119. <https://doi.org/10.1023/A:1022058209073>

² Adams, H., Zinsmeister, N., & Robinson, D. (2020). *Uniswap v2 Core*. Uniswap Labs. <https://uniswap.org/whitepaper.pdf>

³ Adams, H., Zinsmeister, N., Salem, M., Keefer, R., & Robinson, D. (2021). *Uniswap v3 Core*. Uniswap Labs. <https://uniswap.org/whitepaper-v3.pdf>

⁴ Berg, J., Forsythe, R., Nelson, F., & Rietz, T. (2008). Results from a Dozen Years of Election Futures Markets Research. In *Handbook of Experimental Economics Results* (Vol. 1, pp. 742–751). Elsevier. [https://doi.org/10.1016/S1574-0722\(07\)00080-7](https://doi.org/10.1016/S1574-0722(07)00080-7)

⁵ Cowgill, B., Wolfers, J., & Zitzewitz, E. (2009). Using Prediction Markets to Track Information Flows: Evidence from Google. In *Auctions, Market Mechanisms and Their Applications (AMMA 2009)*, LNICST Vol. 14. Springer. https://doi.org/10.1007/978-3-642-03821-1_2

⁶ Levitt, S. D. (2004). Why Are Gambling Markets Organised So Differently from Financial Markets? *The Economic Journal*, 114(495), 223–246. <https://doi.org/10.1111/j.1468-0297.2004.00207.x>

⁷ Gnosis. *Conditional Tokens Framework (CTF) Documentation*. <https://docs.gnosis.io/conditionaltokens/>

⁸ UMA Protocol. *Optimistic Oracle Documentation*. <https://docs.uma.xyz/>

⁹ Leshner, R., & Hayes, G. (2019). *Compound: The Money Market Protocol*. Compound Labs. <https://compound.finance/documents/Compound.Whitepaper.pdf>

¹⁰ SushiSwap. (2020). *MasterChef Contract*. <https://github.com/sushiswap/sushiswap>

¹¹ Piskunov, A. *VIZ: Distributed Ledger Technical Description, Fair DPoS, and Governance*. VIZ-Blockchain. <https://viz-blockchain.github.io/viz-cpp-node/>

¹² Arrow, K. J., Forsythe, R., Gorham, M., Hahn, R., Hanson, R., Ledyard, J. O., et al. (2008). The Promise of Prediction Markets. *Science*, 320(5878), 877–878. <https://doi.org/10.1126/science.1157679>

¹³ Wolfers, J., & Zitzewitz, E. (2004). Prediction Markets in Theory and Practice. *NBER Working Paper* No. 10248 (also *Journal of Economic Perspectives*, 18(2), 107–126). <https://doi.org/10.3386/w10248>

¹⁴ Othman, A., Pennock, D. M., Reeves, D. M., & Sandholm, T. (2013). A Practical Liquidity-Sensitive Automated Market Maker. *ACM Transactions on Economics and Computation*, 1(3), Article 14. <https://doi.org/10.1145/2509413.2509414>

¹⁵ Abernethy, J., Chen, Y., & Vaughan, J. W. (2011). An Optimization-Based Framework for Automated Market-Making. In *Proceedings of the 12th ACM Conference on Electronic Commerce (EC '11)* (pp. 297–306). <https://doi.org/10.1145/1993574.1993621>